

# ALAB 351.3 - Conditionals, Loops, and Lists

---

**Name:** James Sloan

**Course:** Python Essentials (UCI 3052)

**Date:** August 20, 2026

## Repository

---

All files for this lab are in a single GitHub repository:

<https://github.com/santigrey/Python-Essentials>

The scripts for this lab are in `modules/Module 351/ALAB-351.3/` :

<https://github.com/santigrey/Python-Essentials/tree/main/modules/Module%20351/ALAB-351.3>

## Files

| File                             | Part                                          |
|----------------------------------|-----------------------------------------------|
| <code>grade_checker.py</code>    | Part 1 — Conditional Execution                |
| <code>even_sum.py</code>         | Part 1 — Sum of Even Numbers                  |
| <code>list_operations.py</code>  | Part 2 — List Operations                      |
| <code>bubble_sort_demo.py</code> | Part 2 — Bubble Sort                          |
| <code>logic_bits.py</code>       | Part 3 — Logic and Bitwise Operations (bonus) |

## grade\_checker.py

---

*Part 1 — Conditional Execution*

### Code

```
# ALAB 351.3 – Part 1: Conditional Execution
# James Sloan
#
# Asks for a numeric grade and converts it to a letter grade.

grade = float(input("Enter a numeric grade (0-100): "))

# -----
# The if/elif/else chain is checked from the top down, so each branch only
```

```

# needs its lower bound. A grade of 95 matches "grade >= 90" and Python stops
# checking there. That top-down order is also what makes the edge cases work:
# 100 lands in A, 90 lands in A, 60 lands in D, and 0 falls through to F.
# -----
if grade >= 90:
    letter = "A"
elif grade >= 80:
    letter = "B"
elif grade >= 70:
    letter = "C"
elif grade >= 60:
    letter = "D"
else:
    letter = "F"

print(f"Your grade is: {letter}")

# A conditional expression picks one of two values on a single line:
# <value if true> if <condition> else <value if false>
# A, B and C are the passing grades; D and F are not.
passed = letter in ("A", "B", "C")
congrats = "Congratulations, you passed!"
encourage = "Keep studying and try again."

message = congrats if passed else encourage
print(message)

```

## Output

Run with a passing grade:

```

Enter a numeric grade (0-100): 85
Your grade is: B
Congratulations, you passed!

```

Run with a failing grade:

```

Enter a numeric grade (0-100): 55
Your grade is: F
Keep studying and try again.

```

## even\_sum.py

---

### Part 1 — Sum of Even Numbers

#### Code

```
# ALAB 351.3 – Part 1: Sum of Even Numbers
# James Sloan
#
# Adds up every even number from 1 to 50, first with a for loop and then
# with a while loop, to confirm both approaches give the same answer.

# -----
# Version 1: for loop
# range(1, 51) counts 1 through 50. The stop value is not included, so it has
# to be 51 to reach 50.
# A number is even when dividing it by 2 leaves no remainder, so the test is
# number % 2 == 0.
# -----
for_total = 0
for number in range(1, 51):
    if number % 2 == 0:
        for_total = for_total + number

print("Using a for loop:")
print(f"The sum of even numbers from 1 to 50 is {for_total}.")
print()

# -----
# Version 2: while loop
# A while loop does not count for us. We start the counter at 1, test it at
# the top of every pass, and move it forward by hand at the bottom. Leaving
# out that last step is what causes a loop to run forever.
# -----
while_total = 0
number = 1
while number <= 50:
    if number % 2 == 0:
        while_total = while_total + number
    number = number + 1

print("Using a while loop:")
print(f"The sum of even numbers from 1 to 50 is {while_total}.")
```

```

print()

# -----
# Do both versions agree?
# Yes – both print 650.
#
# The for loop is the clearer of the two for me. range() handles the starting
# value, the stop value, and the counting all on one line, so there is less to
# keep track of and no way to forget to move the counter forward. The while
# loop needs three separate pieces – start the counter, test it, increase it –
# and the answer only comes out right if all three agree with each other.
# -----
print(f"Both loops produced the same result: {for_total == while_total}")

```

## Output

```

Using a for loop:
The sum of even numbers from 1 to 50 is 650.

Using a while loop:
The sum of even numbers from 1 to 50 is 650.

Both loops produced the same result: True

```

## list\_operations.py

### Part 2 — List Operations

## Code

```

# ALAB 351.3 – Part 2: List Operations
# James Sloan
#
# Creates a list of integers and walks through the common list operations,
# printing the list after each step so the change is visible.
#
# -----
# Step 1: Create the list and print it.
# A list is written in square brackets with the items separated by commas.
# -----
numbers = [42, 7, 19, 3, 25]

```

```

print("Original list:", numbers)
print()

# -----
# Step 2: sorted() is a built-in function that hands back a NEW sorted list
# and leaves the original alone. Printing the original again afterwards is
# what proves it was not changed.
# -----
print("Sorted copy from sorted():", sorted(numbers))
print("Original list, still unchanged:", numbers)
print()

# -----
# Step 3: .sort() is a list method that sorts the list in place. It changes
# the list itself and hands back nothing. That is the key difference from
# sorted() in Step 2.
# -----
numbers.sort()
print("After numbers.sort(), sorted in place:", numbers)
print()

# -----
# Step 4: .append() adds one item onto the end of the list.
# -----
numbers.append(50)
print("After appending 50:", numbers)
print()

# -----
# Step 5: .remove() deletes the first item matching the value given.
# Removing by position would be numbers.pop(0) instead, which removes by index.
# -----
numbers.remove(19)
print("After removing the value 19:", numbers)
print()

# -----
# Step 6: .reverse() flips the order of the list in place.
# -----
numbers.reverse()
print("After reversing:", numbers)

```

## Output

```
Original list: [42, 7, 19, 3, 25]

Sorted copy from sorted(): [3, 7, 19, 25, 42]
Original list, still unchanged: [42, 7, 19, 3, 25]

After numbers.sort(), sorted in place: [3, 7, 19, 25, 42]

After appending 50: [3, 7, 19, 25, 42, 50]

After removing the value 19: [3, 7, 25, 42, 50]

After reversing: [50, 42, 25, 7, 3]
```

## bubble\_sort\_demo.py

---

### *Part 2 — Bubble Sort*

## Code

```
# ALAB 351.3 – Part 2: Bubble Sort
# James Sloan
#
# Sorts a list of integers by hand using bubble sort, printing the list after
# every pass so the progress of the sort is visible.
#
# How bubble sort works:
#   Walk along the list comparing each pair of neighbours. If the left one is
#   bigger than the right one, swap them. Doing that once carries the largest
#   remaining number all the way to the end of the list – it "bubbles up".
#   Repeat the walk and the next-largest number settles into place, and so on
#   until nothing is left out of order.

numbers = [64, 25, 12, 22, 11]
n = len(numbers)

print("Starting list:", numbers)
print()

# -----
```

```

# The OUTER loop counts the passes. Each completed pass locks one more number
# into its final position at the end of the list, and n - 1 passes are enough
# to place every number (the last one is left over and already correct).
# -----
for outer in range(n - 1):

    # -----
    # The INNER loop does one walk along the list, comparing neighbours.
    #   n - 1 because we compare a pair, so we stop one short of the end -
    #       otherwise numbers[inner + 1] would run off the list.
    #   - outer because the last `outer` numbers were settled by the earlier
    #       passes, so there is no reason to check them again.
    # -----
    for inner in range(n - 1 - outer):

        # A swap happens only when the left number is bigger than the right
        # one, which means the pair is out of order.
        if numbers[inner] > numbers[inner + 1]:
            # Python swaps two values in a single line - the right-hand side
            # is worked out first, so no temporary variable is needed.
            numbers[inner], numbers[inner + 1] = numbers[inner + 1], numbers[inner]

    # Printed once per pass of the outer loop, to show the sort's progress.
    print(f"After pass {outer + 1}: {numbers}")

print()
print("Final sorted list:", numbers)

```

## Output

```

Starting list: [64, 25, 12, 22, 11]

After pass 1: [25, 12, 22, 11, 64]
After pass 2: [12, 22, 11, 25, 64]
After pass 3: [12, 11, 22, 25, 64]
After pass 4: [11, 12, 22, 25, 64]

Final sorted list: [11, 12, 22, 25, 64]

```

## logic\_bits.py

---

### Part 3 — Logic and Bitwise Operations (bonus)

#### Code

```
# ALAB 351.3 – Part 3: Logic and Bitwise Operations (bonus)
# James Sloan
#
# Two short demonstrations: the logical operators and/or/not on boolean values
# typed by the user, and the bitwise operators on the integers 5 and 3.

# -----
# Part A: Logical operators
# input() always hands back text, so the typed answer has to be turned into a
# real True or False. .lower() lets "True", "true" and "TRUE" all work, and the
# "in" test accepts either the word or the digit 1. Anything else counts as False.
# -----
print("Logical operators")
print("=====")

first_text = input("Enter the first value (True/False or 1/0): ").strip()
second_text = input("Enter the second value (True/False or 1/0): ").strip()

first = first_text.lower() in ("true", "1")
second = second_text.lower() in ("true", "1")

print()
print(f"first = {first}")
print(f"second = {second}")
print()

print(f"first and second = {first and second}")    # True only when BOTH are True
print(f"first or second = {first or second}")      # True when EITHER is True
print(f"not first      = {not first}")             # flips a value to its opposite
print(f"not second     = {not second}")
print()

# -----
# Part B: Bitwise operators
# These work on the individual bits inside a number rather than on its value.
# bin() shows those bits, with the 0b in front marking a binary number.
# bin() does not pad with leading zeros, so 3 prints as 0b11 rather than 0b011.
```



```

# Lining the two up by hand shows what the comparisons below are working on:
#   5 -> 101
#   3 -> 011
# -----
print("Bitwise operators")
print("=====")

a = 5
b = 3

print(f"a = {a} -> {bin(a)}")
print(f"b = {b} -> {bin(b)}")
print()

print(f"a & b = {a & b} -> {bin(a & b)}")      # 1 only where BOTH have a 1
print(f"a | b = {a | b} -> {bin(a | b)}")      # 1 where EITHER has a 1
print(f"a ^ b = {a ^ b} -> {bin(a ^ b)}")      # XOR: 1 where they DIFFER
print(f"~a = {~a} -> {bin(~a)}")              # flips every bit (see note below)
print(f"a << 1 = {a << 1} -> {bin(a << 1)}")    # shift left doubles the number
print(f"a >> 1 = {a >> 1} -> {bin(a >> 1)}")    # shift right halves it

# Note on ~ : Python has no fixed number of bits per integer, so it prints the
# flipped result as a negative number instead of a raw row of bits. ~5 is -6.
# Note on >> : shifting right throws away the bit that falls off the end, so
# 5 >> 1 is 2 and not 2.5 - the remainder is dropped, not rounded.

```

## Output

```

Logical operators
=====
Enter the first value (True/False or 1/0): True
Enter the second value (True/False or 1/0): False

first = True
second = False

first and second = False
first or second = True
not first = False
not second = True

Bitwise operators

```

=====

a = 5 -> 0b101

b = 3 -> 0b11

a & b = 1 -> 0b1

a | b = 7 -> 0b111

a ^ b = 6 -> 0b110

~a = -6 -> -0b110

a << 1 = 10 -> 0b1010

a >> 1 = 2 -> 0b10

---